

Intelligent Candidate Discovery & Rank

Team Antigravity

An explicit, weighted hybrid scoring architecture.

Architecture & Methodology

Our ranker is built to run entirely locally, well within the 5-minute compute budget. We avoid heavy LLM calls at runtime by using a pre-computed local embedding model.

The scoring formula is explicitly weighted into 3 pillars:

1. Semantic Match (40% Weight)

- Uses all-MiniLM-L6-v2 via sentence-transformers.
- Computes Cosine Similarity between the JD and Candidate Summary + Experience.

2. Rule-Based Heuristics (30% Weight)

- Extracts vector DB signals (Pinecone, Milvus, Qdrant) from role descriptions.
- Penalizes pure academic research and consulting backgrounds as per the JD.

3. Behavioral Signals (30% Weight)

- Factors in Recruiter Response Rate, GitHub Activity, and Interview Completion Rate.
- Applies penalties to inactive or dormant accounts.

Honeypot Detection & Avoidance

Keyword stuffing is a major risk in naive recruitment systems.

We proactively filter out candidates with impossible credentials:

- 'Expert' proficiencies with 0 months of duration.
- Titles that don't match the skills listed (e.g. Marketing Manager with PyTorch).

By enforcing a hybrid architecture, our semantic layer ensures that the context of the skills matters, not just their presence. This robustly guards against the exact trap explicitly mentioned in the Job Description.